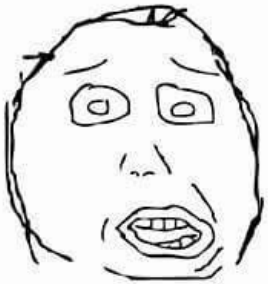
P05

DOCUMENTACIÓN DE CÓDIGO Y SUGERENCIA DE TIPOS

"Code is more often read than written"

–Guido van Rossum
(creador de Python)

Programming alone



```
c = a + b;
```

Programming while someone watches

```
/// <summary>
/// A function, that adds two numbers
/// </summary>
/// <param name="a">First number</param>
/// <param name="b">Second number</param>
/// <returns>Sum of a and b</returns>
```

```
private int Add(int a, int b)
{
   // This line adds two ints
   return a + b;
}
```

DOCSTRINGS

```
>>> help(str)
Help on class str in module builtins:

class str(object)
 |  str(object='') -> str
 |  str(bytes_or_buffer[, encoding[, errors]]) -> str
 |
 |  Create a new string object from the given object. If encoding or
 |  errors are specified, then the object must expose a data buffer
 |  that will be decoded using the given encoding and error handler.
 |  Otherwise, returns the result of object.__str__() (if defined)
 |  or repr(object).
 |  encoding defaults to sys.getdefaultencoding().
 |  errors defaults to 'strict'.
 # Truncated for readability
```

La documentación del código Python se centra en los docstrings. Son cadenas que, cuando se configuran correctamente, pueden ayudar a los usuarios y a nosotros mismos con la documentación de proyectos. Junto con los docstrings, Python también tiene la función incorporada help() que imprime el docstring de los objetos en la consola.

DOCSTRINGS EN CLASES

```
class SimpleClass:
    """Class docstrings go here."""

    def say_hello(self, name: str):
        """Class method docstrings go here."""

        print(f'Hello {name}')
```

Usaremos el estilo de docstrings de NumPy:

https://numpydoc.readthedocs.io/en/latest/format.html

DOCSTRINGS EN CLASES

```
class Animal:

    says_str = "A {name} says {sound}"

    def __init__(self, name, sound, num_legs=4):

        self.name = name
        self.sound = sound
        self.num_legs = num_legs

    def says(self, sound=None):

        if self.sound is None and sound is None:
            raise NotImplementedError("Silent Animals are not supported!")

        out_sound = self.sound if sound is None else sound
        print(self.says_str.format(name=self.name, sound=out_sound))
```

DOCSTRINGS EN CLASES

```
class Animal:
    """
    A class used to represent an Animal

    Attributes
    ----------
    says_str : str
        a formatted string to print out what the animal says
    name : str
        the name of the animal
    sound : str
        the sound that the animal makes
    num_legs : int
        the number of legs the animal has (default 4)

    Methods
    -------
    says(sound=None)
        Prints the animals name and what sound it makes
    """

    says_str = "A {name} says {sound}"
```

DOCSTRINGS EN CLASES

```
def __init__(self, name, sound, num_legs=4):
        """
        Parameters
        ----------
        name : str
            The name of the animal
        sound : str
            The sound the animal makes
        num_legs : int, optional
            The number of legs the animal (default is 4)
        """

        self.name = name
        self.sound = sound
        self.num_legs = num_legs
```

DOCSTRINGS EN CLASES

```
def says(self, sound=None):
        """Prints what the animals name is and what sound it makes.

        If the argument `sound` isn't passed in, the default Animal
        sound is used.

        Parameters
        ----------
        sound : str, optional
            The sound the animal makes (default is None)

        Raises
        ------
        NotImplementedError
            If no sound is set for the animal or passed in as a
            parameter.
        """

        if self.sound is None and sound is None:
            raise NotImplementedError("Silent Animals are not supported!")

        out_sound = self.sound if sound is None else sound
        print(self.says_str.format(name=self.name, sound=out_sound))
```

Lo veremos en el tema de excepciones

DOCSTRINGS EN SCRIPTS

```python
import argparse

import pandas as pd


def get_spreadsheet_cols(file_loc, print_cols=False):

    file_data = pd.read_excel(file_loc)
    col_headers = list(file_data.columns.values)

    if print_cols:
        print("\n".join(col_headers))

    return col_headers


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        'input_file',
        type=str,
        help="The spreadsheet file to pring the columns of"
    )
    args = parser.parse_args()
    get_spreadsheet_cols(args.input_file, print_cols=True)


if __name__ == "__main__":
    main()
```

DOCSTRINGS EN SCRIPTS

```
"""Spreadsheet Column Printer

This script allows the user to print to the console all columns in the
spreadsheet. It is assumed that the first row of the spreadsheet is the
location of the columns.

This tool accepts comma separated value files (.csv) as well as excel
(.xls, .xlsx) files.

This script requires that `pandas` be installed within the Python
environment you are running this script in.

This file can also be imported as a module and contains the following
functions:

    * get_spreadsheet_cols - returns the column headers of the file
    * main - the main function of the script
"""

import argparse

import pandas as pd
```

DOCSTRINGS EN SCRIPTS

```
def get_spreadsheet_cols(file_loc, print_cols=False):
    """Gets and prints the spreadsheet's header columns

    Parameters
    ----------
    file_loc : str
        The file location of the spreadsheet
    print_cols : bool, optional
        A flag used to print the columns to the console (default is
        False)

    Returns
    -------
    list
        a list of strings used that are the header columns
    """

    file_data = pd.read_excel(file_loc)
    col_headers = list(file_data.columns.values)

    if print_cols:
        print("\n".join(col_headers))

    return col_headers
```

FICHEROS ADICIONALES DE DOCUMENTACIÓN

- **Readme**: Un breve resumen del proyecto y su propósito. Incluye cualquier requisito especial para la instalación o el funcionamiento del proyecto. En GitHub, se suele llamar README.md y tiene un formato definido.

- **examples.py**: Un archivo de script en Python que ofrece ejemplos sencillos de cómo utilizar el proyecto.

- **License** (Licencia): Un archivo de texto plano que describe la licencia que utiliza el proyecto. Especialmente para proyectos de código abierto, considerad añadir esto.

- **docs**: Una carpeta que contiene documentación adicional y complementaria.

SUGERENCIA DE TIPOS

SUGERENCIA DE TIPOS

- Python es un lenguaje de tipado dinámico. Esto significa que el intérprete de Python realiza la comprobación de tipos sólo durante la ejecución del código, y que el tipo de una variable puede cambiar a lo largo de su vida.
- Otro término que se utiliza a menudo cuando se habla de Python es *duck typing*. Este apodo proviene de la frase "si camina como un pato y grazna como un pato, entonces debe ser un pato".
- Duck typing es un concepto relacionado con la tipificación dinámica, en la que el tipo o la clase de un objeto es menos importante que los métodos que define. Con el tipado dinámico no se comprueban los tipos. En su lugar, se comprueba la presencia de un determinado método o atributo.
- La sugerencia de tipos (Type hinting), que introdujo en Python 3.5, es una solución formal para indicar estáticamente el tipo de un valor dentro del código Python.
- He aquí un ejemplo de cómo añadir información de tipo a una función:

```
def saludar(nombre: str) -> str:
    return "Hola, " + nombre
```

VARIABLE, FUNCIONES Y CLASES

```python
a: int = 3
b: float = 2.4
c: bool = True
d: list = ["A", "B", "C"]
e: dict = {"x": "y"}
f: set = {"a", "b", "c"}
g: tuple = ("name", "age", "job")

def add_numbers(x: int, y: int= 100) -> int:
  return x + y

class Person:
  first_name: str = "John"
  last_name: str = "Does"
  age: int = 31
```

LISTAS Y SECUENCIAS

```python
from typing import List

def sum_of_list(l: List[float]):
    return sum(l)


from typing import Sequence

def process_sequence(data: Sequence) -> None:
    for item in data:
        print(item)

# Works with any iterables
process_sequence([1, 2, 3]) # list
process_sequence({1, 2, 3}) # set
process_sequence({1: "a", 2: "b", 3: "c"}) # dict
process_sequence((1, 2, 3)) # tuple
process_sequence("123") # string
```

Un objeto de tipo *Sequence* se refiere a cualquier cosa que pueda ser indexada, como una lista, una tupla o una cadena, o incluso estructuras más complejas como una lista de objetos o una tupla de listas de tuplas. Esto nos permite pasar cualquier objeto iterable, y la función funcionará correctamente.

OPCIONALES

```python
from typing import Optional

def greet(name: Optional[str]) -> str:
    if name is None:
        return "Hello, stranger!"
    else:
        return f"Hello, {name}!"

print(greet("Alice"))   # Output: Hello, Alice!
print(greet(None))      # Output: Hello, stranger!
```

UNIONES

```python
from typing import Union

def parse_input(data: Union[int, str]) -> Union[int, str]:
    if isinstance(data, int):
        return int(data)
    elif isinstance(data, str):
        try:
            return int(data)
        except ValueError:
            return data

result1 = parse_input(123)
print("Parsed result 1:", result1)
# Output: Parsed result 1: 123 (as an integer)

result2 = parse_input("456")
print("Parsed result 2:", result2)
# Output: Parsed result 2: 456 (as an integer)

result3 = parse_input("abc")
print("Parsed result 3:", result3)
# Output: Parsed result 3: abc (remains as a string)
```

OBJETOS DE LA CLASE ACTUAL

```
from typing import Self

class Person:
    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age

    def birthday(self) -> Self:
        self.age += 1
        return self

p = Person("Alice", 30)
p.birthday()  # Person instance with age 31
```

OBJETOS DE OTRAS CLASES

```
class B: def __init__(self, name: str):
        self.name = name

class A:
    def __init__(self, b: 'B'):
        self.b = b

    def get_b(self) -> 'B':
        return self.b

b_instance = B("Object B")
a_instance = A(b_instance)
print(a_instance.get_b().name)
```

SUGERENCIA DE TIPOS

```
name: str = "Phil"
age: int = 29
height_metres: float = 1.87
loves_python: bool = True


from typing import Union

name: str = "Phil"
age: int = 29
height_metres: Union[int, float] = 1.87

> mypy app.py
Success: no issues found in 1 source file
```

Mypy es un comprobador opcional de tipado estático para Python que pretende combinar las ventajas del tipado dinámico (o "duck") y el tipado estático.

Lo instalamos con:

```
python -m pip install mypy
```

SUGERENCIA DE TIPOS

```
name: str = 29
age: int = 1.87
height_metres: float = "Phil"

> mypy app.py
app.py:1: error: Incompatible types in assignment
(expression has type "int", variable has type "str")
app.py:2: error: Incompatible types in assignment
(expression has type "float", variable has type "int")
app.py:3: error: Incompatible types in assignment
(expression has type "str", variable has type "float")
Found 3 errors in 1 file (checked 1 source file)
```

SUGERENCIA DE TIPOS

```
def find_movie(search_term: str, movies: \
          list[Movie]) -> Union[Movie, None]:
    for title, director, year in movies:
        if title == search_term:
            return (title, director, year)

> mypy app.py
app.py:5: error: Missing return statement
Found 1 error in 1 file (checked 1 source file)
```

SUGERENCIA DE TIPOS

```
def find_movie(search_term: str, movies: \
          list[Movie]) -> Union[Movie, None]:
    for title, director, year in movies:
        if title == search_term:
            return (title, director, year)

    return None

> mypy app.py
Success: no issues found in 1 source file
```

EJERCICIOS DE PROGRAMACIÓN ORIENTADA A OBJETOS: CLASES

EJERCICIO 1

- Crea una clase llamada "BankAccount" que represente una cuenta bancaria simple.
- Debe tener atributos para el nombre y el saldo del titular de la cuenta.
- Implementa métodos para depositar y retirar dinero, garantizando al mismo tiempo que el saldo no pueda volverse negativo.
- Prueba tu clase BankAccount creando una cuenta, depositando dinero y realizando retiros.

EJERCICIO 2

- Crea la clase Coche que contenga las siguientes propiedades (no públicas):
  - matrícula (string)
  - marca (string)
  - kilometros_recorridos (float)
  - gasolina (float)

- La clase tendrá un método llamado avanzar() que recibirá como argumento el número de kilómetros a conducir y sumará los kilómetros recorridos al valor de la propiedad kilometros_recorridos.

- El método también restará al valor de gasolina el resultado de los kilómetros multiplicado por 0'1. La clase también contendrá otro método llamado repostar() que recibirá como argumento los litros introducidos que deberán sumarse a la variable gasolina.

- Por último, será necesario controlar que el método avanzar nunca obtendrá un número negativo en la gasolina. En dicho caso, deberá mostrar el siguiente mensaje: "Es necesario repostar para recorrer la cantidad indicada de kilómetros".

- Crea el método __str__ que permita imprimir los detalles de un coche en el siguiente formato:
  - Coche Seat (1234BCD). Kms recorridos: 10, Depósito: 24.0 litros

EJERCICIO 3

- Modifica la clase Coche para que sea capaz de almacenar los kms recorridos por los coches de cada marca
- Para eso, crea el método kms_por_marca(marca) y los atributos necesarios
- Si se piden los kms de los coches de una marca de la cual no exista ningún coche, se mostrará un mensaje de error

EJERCICIO 4

- Crea la clase Persona, que almacene el nombre y apellido de una persona, así como qué coche tiene (objeto Coche), teniendo en cuenta que el constructor no asignará un coche a una persona.

- Para asignar un coche a una persona, crearemos el método recibir_coche(coche). Pero ten en cuenta que una persona solo puede tener un coche 🙅🏻.

- Crea el método vender_coche(compador), que permita a una persona vender un coche (si lo tiene) a un comprador que no tiene coche

- Crea el método __str__() que permita imprimir los datos de una persona y de su coche

EJERCICIOS EXTRA

- Modifica la clase Coche para que pueda almacenar quién es su dueño
- Modifica la clase Persona para que pueda tener más de un coche